

Übungsblatt 7

Ü 7.1 Wiederholung Interprozesskommunikation: Pipes

Erstellen Sie ein C-Programm `ipc.c`, welches über einen Kind-Prozess verfügt und mittels des Pipe-IPC-Kommunikationsverfahrens Nachrichten mit dem Hauptprozess austauscht. Erfüllen Sie folgende Programmschritte:

- Erstellen Sie ein Main-Programm, in dem die Prozesse verwaltet werden. Starten Sie alle benötigten Prozesse (Eltern und Kind).
- Erstellen Sie eine Variable, die Sie als Pipe verwenden wollen, und finden Sie Methoden, um auf die besagte Pipe zu lesen bzw. schreiben zu können.
- Schreiben Sie vom Kind-Prozess eine beliebige Anzahl von Nachrichten an den Hauptprozess. Die Ausgabe könnte wie folgt aussehen:

```
write from 764: hello
write from 764: hello 2
write from 764: hello 3
message #1 read by 763: hello
message #2 read by 763: hello 2
message #3 read by 763: hello 3
```

Ü 7.2 Interprozesskommunikation: Message Queues

Machen Sie sich mit folgenden Funktionen vertraut: `msgget`, `msgsnd`, `msgrcv`, `msgctl` und schreiben Sie ein Programm `msq.c`, das Nachrichten mittels des Message-Queue-(IPC)-Paradigmas zwischen zwei Prozessen austauscht.

- Gestalten Sie ein Hauptprogramm welches beide Prozesse implementiert und Queues anlegt
- Verwenden Sie die folgende Struktur, um Daten auszutauschen. Verwenden Sie zum Senden und Empfangen die oben erwähnten Funktionen.

```
struct message {
    long mtype;
    char mtext[8192];
};
```

Eine mögliche Ausgabe könnte wie folgt aussehen:

```
sending <Hello parent from child using messageq> in 1596
received <Hello parent from child using messageq> in 1592
```

Ü 7.3 Threads – Grundlagen / Erzeugung

In der Vorlesung wurden Threads ausführlich besprochen.

- a) Was ist der Unterschied zwischen Threads und Prozessen?
- b) Was sind POSIX Threads? (Hinweis: `man pthreads`) Erklären Sie folgende Befehle bzw. Konstrukte in diesem Zusammenhang:

```
pthread_t; pthread_self; pthread_create; pthread_exit; pthread_join;  
pthread_detach; pthread_mutex_t; pthread_mutex_lock;  
pthread_mutex_unlock; pthread_cond_init; pthread_cond_destroy;  
pthread_cond_signal; pthread_cond_broadcast; pthread_cond_wait.
```

Was müssen Sie eventuell (abhängig von Ihrer glibc-Version) unter Verwendung der „pthread“-Bibliothek beim Kompilieren bzw. Linken beachten?

Schreiben Sie ein C-Programm namens `threadcountdown.c`, das n Threads erzeugt. Die Anzahl n hängt von der Anzahl der beim Aufruf übergebenen Kommandozeilenparameter p_1 p_2 ... p_n ab. Jeder Thread soll einen Countdown durchführen, wobei der i -te Thread im Sekundentakt von p_i bis 0 herunterzählt. Die Ausgabe des Countdowns der einzelnen Threads soll am Bildschirm erfolgen und zusätzlich die aktuelle Thread-ID enthalten. Das folgende Beispiel veranschaulicht die geforderte Funktionsweise.

```
Aufruf: ./threadcountdown 3 5 4  
Ausgabe: Thread 3084327824: will count from 3 to 0  
          Thread 3084327824: 3  
          Thread 3075935120: will count from 5 to 0  
          Thread 3075935120: 5  
          Thread 3067542416: will count from 4 to 0  
          Thread 3067542416: 4  
          Thread 3084327824: 2  
          Thread 3075935120: 4  
          Thread 3067542416: 3  
          Thread 3084327824: 1  
          Thread 3075935120: 3  
          Thread 3067542416: 2  
          Thread 3084327824: 0  
          Thread 3075935120: 2  
          Thread 3067542416: 1  
          Thread 3084327824: finished...  
          Thread 3067542416: 0  
          Thread 3075935120: 1  
          Thread 3075935120: 0  
          Thread 3067542416: finished...  
          Thread 3075935120: finished...
```

Ü 7.4 Prozesse vs. Threads

Schreiben Sie zwei C-Programme `forkmatrix.c` und `threadmatrix.c`, welche einmal mit Hilfe von Prozessen (`fork`) und einmal mit Hilfe von Threads (`pthread`), mit N Prozessen bzw. Threads eine mit Zufallszahlen gefüllte $N \times N$ -Matrix durchlaufen und die **jeweilige Spaltensumme** berechnen (z.B. bei $N=100$ berechnet der 50. Prozess bzw. Thread die Summe der 50. Spalte in der 100×100 -Matrix).

Vergleichen Sie die Laufzeiten beider Programme (z.B. `time ./programmname`). Bestimmen Sie den Speedup und erklären Sie, wie dieser erreicht wird bzw. wodurch sich dieser bemerkbar macht.

Hinweis: Wählen Sie N ausreichend groß und achten Sie auf die maximal erlaubte Anzahl der Prozesse bzw. Threads in Ihrem System. Nehmen Sie ggf. Anpassungen vor. Wie kann dies erreicht werden? (Siehe `ulimit`)

Bonusaufgabe (+1 MP an der Tafel): Implementieren Sie die beiden Programme mit den Matrizen sowohl im Stack (`int[][]`) als auch im Heap (`int**`). Vergleichen Sie die Laufzeiten und erklären Sie die Unterschiede. Probieren Sie es auch mit kleinen und besonders großen N .

Ü 7.5 Deadlock Analyse

Ein Programm nutzt vier Threads (T_1, T_2, T_3, T_4), die jeweils unterschiedliche Ressourcen anfordern. Die Ressourcen werden als ein Integer-Array mit vier Werten dargestellt (R_1 bis R_4). Jede Ressource kann nur von einem Thread gleichzeitig beansprucht werden und jeder Zugriff auf eine Ressource dauert 100 Millisekunden, was durch `sleep(0.1)` simuliert wird. Sichern Sie den Zugriff auf jede Ressource separat ab.

Die Ressourcenanforderungen sind wie folgt:

- **T_1 benötigt die Ressourcen R_1, R_2 und R_3**
- **T_2 benötigt die Ressourcen R_3 und R_4**
- **T_3 benötigt die Ressourcen R_1 und R_4**
- **T_4 benötigt die Ressourcen R_1 und R_4**

Alle Threads laufen parallel und greifen in zufälliger Reihenfolge auf die ihnen zugeteilten Ressourcen zu. Dabei kann es zu einem Deadlock kommen, bei dem sich zwei oder mehr Threads gegenseitig blockieren und das Programm nicht mehr weiterläuft.

Aufgabenstellung:

1. **Analyse:** Führen Sie das gegebene Programm aus und beobachten Sie, wann und warum es zu einem Deadlock kommt. Erklären Sie die Ursache des Deadlocks anhand der Coffman-Bedingungen.
2. **Modifikation:** Ändern Sie den Code so, dass der Deadlock vermieden wird. Sie können eine oder mehrere der folgenden Strategien anwenden:
 - Konsistente Sperrreihenfolge (Lock Ordering)
 - Verwendung von `try_lock`, um blockierte Ressourcen zu erkennen